

How to Den

Den.pm is a Perl module to convert denaturant titration melt data from “timed” titrations. Below is a sample test script, test_den.pl, to process a titration that contains two different segments of titration points, which were collected in two data files on a fluorimeter, 072808A.PRN and 072808B.PRN:

```
use Den;
my $d = Den -> new()
    -> open_parameter_file('den_parameter_file.txt')
    -> conjoin_files('072808A.PRN', '072808B.PRN', 'conjoined.txt')
    -> calc_den()
    -> extract_signal()
    -> save_data('072808_MELT.txt');
```

A key file is the parameter file

This file is opened using the method, open_parameter_file(\$some_filename). Here is an example, den_parameter_file.txt, used in the case above:

```
CUVETTE_VOLUME    2000
TITRANT_CONC       7.02
CUVETTE_CONC       0
TITRATION_PROGRAM      30_25_48_64
TIME_INIT 106
TIME_PER_BIN       106
TIME_START         0
TIME_END  8374
TIME_OFFSET        3180
```

This parameter file is in a KEY tab value format. It always uses tab. These keys refer to properties in the Den module, where the parameters are stored. All keys may be accessed by the all lower case method gets and sets, i.e.:

```
my $tp = $d -> titration_program();
$d -> titration_program($tp);
```

This would get the variable, \$tp, in the former case and set the property to \$tp in the latter. Sure you could just set all those parameters in the test_den.pl script this way if you’d like to do it that way and not open the parameter file at all.

The above script, test_den.pl, is fancy where two segments were collected. TITRATION_PROGRAM, states these two segments are as follows in its shorthand form, i.e., 30_25_48_64:

SEG	PTS.	VOLUME (UL)
=====		
SEG1	30	25
SEG2	25	64
=====		

NOTE the TITRATION_PROGRAM must separate the values for the segments using only an underscore and no spaces or other characters—only numbers and underscores.

CUVETTE_VOLUME, TITRANT_CONC, and CUVETTE_CONC should be self explanatory.

Extracting the signal with the `extract_signal` method

`TIME_INIT` is the time prior to the first titration operation (usually where the cuvette is in 0 M denaturant or something like that. `TIME_INIT` is generally the same as the `TIME_PER_BIN`, which is the time taken between titration steps.

The parameters `TIME_START` and `TIME_END` are basically unnecessary, although originally they allowed the algorithm to exclude data before time start and after time end during the signal extraction process.

The `extract_signal` method only requires these parameters (`TIME_PER_BIN` and `TIME_INIT`) and some raw data to work off of to extract the signal. So maybe you'd like to open up a file of raw data first, using `open_raw_data($some_file)`:

```
use Den;
my $d = Den -> new()
    -> open_parameter_file('den_parameter_file.txt')
    -> open_raw_data('072808A.PRN')
    -> extract_signal();
```

The method can take the parameters for init and bin times in the method as arguments, but this is not needed if you set them in the parameter file or separately above:

```
$d -> extract_signal($time_init, $time_per_bin);
my @signal_array = $d -> signal();
```

The signal is then stored into the array property, `signal`, accessed simply as above.

Conjoining files for two different segments

```
conjoin_files($file_1_to_join, $file_2_to_join, $conjoined_filename,
    $time_offset, $delimiter);
my @raw = $d -> raw_data();
```

Here `TIME_OFFSET` is important to time shift the second file when stitching them together with the `conjoin_files` method. The offset may be set in the parameter file or as an argument here. On yeah and the delimiter can be set with `$d -> delimiter(chr(9))` if you like tab delimited format, although tab is default. The raw data is accessed using the `raw_data` property.

Calculating the denaturant concentration during the titration

If the `TITRATION_PROGRAM` was set along with `CUVETTE_VOLUME`, `TITRANT_CONC`, and `CUVETTE_CONC`, then the denaturant concentrations can be calculated using the `calc_den` method:

```
$d -> calc_den($titrant_conc, $cuvette_conc, $cuvette_volume, $titration);
my @denaturant_array = $d -> den();
```

Remember that those args are not needed if the parameters were set in the parameter file. So simply `calc_den()` will do. To get the data, use the `den` property.

Accessing the data

You can get the data from the module using the data property:

```
$d -> calc_den();  
$d -> extract_signal();  
my @data = $d -> data();
```

Example for a simple, single-segment melt

For this case, the data is contained in a single file, 01.PRN.

```
use Den;  
my $d = Den -> new()  
    -> open_parameter_file('den_parameter_file.txt')  
    -> open_raw_data('01.PRN')  
    -> calc_den()  
    -> extract_signal()  
    -> save_data('01_MELT.txt');
```

This is the parameter file, den_parameter_file.txt, for the simple case above.

```
CUVETTE_VOLUME    2000  
TITRANT_CONC      7.02  
CUVETTE_CONC      0  
TIME_INIT 29.5  
TIME_PER BIN      29.5  
TITRATION_PROGRAM      153_25
```

How to Hoh

Now you can be a data farmer. The Hoh module (`Hoh.pm`) can manipulate datasets to make an output file, table, or hash of hashes (a hoh). Use as a typical Perl module:

```
use Hoh;
$hoh = Hoh->new();
```

Data can be input as text files that are delimited by tabs, commas, or whatever. Be sure to set the property of the delimiter if not using tabs, i.e., tab is default:

```
$hoh -> delimiter(',');
```

File format for your data should have the first row as the list of data columns. Each column name must be unique single word entries or data will be overwritten by the last instance of a repeated name. These column names are keys to the hoh (see below). Don't use spaces in the names. All Caps is a good convention at this point. Then each row starts with a unique name. Consider the file, `my_data.txt`:

```
MY_KEYS  CNAME1 CNAME2 CNAME3 ...
UNIQUE_1 Bob   Harry Moe
UNIQUE_2 George John  12345
UNIQUE_AB Pi    Beta  Kappa
...
```

Also if the data in your table has spaces in the individual entries, you should either use quotes around the entry or use underscores. For example, the entry, Mr. Rogers, should be either `Mr._Rogers` or `"Mr. Rogers"` to be handled properly.

A file may be opened to make a hoh using the `file_to_hoh` method:

```
my %opened_hoh = $hoh -> file_to_hoh('my_data.txt');
```

`open_hoh` is basically like `file_to_hoh`, except `open_hoh` returns the module and can be used like this:

```
my $hoh_obj = $hoh -> new() -> open_hoh('my_data.txt');
```

You could set the filename and delimiter those properties prior to opening if you like.

```
$hoh -> file('my_data.txt');
$hoh -> delimiter(',');
$hoh -> open_hoh();
```

Now you may access data in the hoh in this way, which retrieves 'George' in the example below in `$datum`:

```
my ($row, $col) = ('UNIQUE_2', 'CNAME1');
my $datum = $opened_hoh{ $row }{ $col };
```

If you want to resave the hoh after some modification, then you must place it back in the `hoh` property and use the `save` method.

```
$opened_hoh{ $row }{ $col } = $something_else;
$hoh -> hoh(%opened_hoh);
$hoh -> save('my_new_data.txt');
```

You can save certain specified column slices of the data in a specific order by using the `print_order` property:

```
$hoh -> print_order(qw(CNAME2 CNAME1));
```

In the above example, CNAME2 will be printed before CNAME1. A print order array can also be loaded from a file, where the file contains a whitespace separated list of single word column names to print in the exact order listed.

```
$hoh -> open_print_order_file($my_filename);
```

Without `print_order` defined, the whole hoh is printed in alphabetical order.

To join two files together where row keys may overlap can be accomplished either by joining two hohs together of similar data structure or opening files. The two hashes must be passed as references to hashes, of course:

```
my $href_a = \%hash_a;  
my $href_b = \%hash_b;  
$hoh -> join_hohs($href_a, $href_b);
```

The joined Hoh can be accessed using:

```
my %hoh = $hoh -> hoh();
```

I think that Santa says the same thing around Christmas. Also files can be opened and joined and saved using a single method:

```
$hoh -> join_hohs($hoh_file_a, $hoh_file_b, $joined_hoh_file_out,  
$print_order_file);
```

Don't worry if you leave out an output file name. The default output file is `outfile.txt`. Also remember that the print order may be set in the property `print_order` as a list or opened separately. See example below. You can even sort the rows alphabetically by a specified column name using the `sort_by` property:

```
$hoh -> sort_by('CNAME2');
```

If you don't specify the `sort_by` property the data are output by alphabetical order using the row key column.

Example script that joins two files together, and then joins to a third file. The print order is defined by a `qw()` list or any array:

```
use Hoh;
my $h = Hoh->new();
$h -> sort_by('AANUM');

my @list = qw(AANUM DDGdD70 DDGdD40ERR DDGDD40 DDGDD70ERR);

$h -> print_order(@list);
$h -> join_files('ddgdd40.txt', 'ddgdd70.txt');

#Note default output file from join_files is outfile.txt

@list = qw (AANUM DDGNI DDGNIERR DDGDD40 DDGDD40ERR DDGDD70 DDGDD70ERR);

$h -> print_order(@list);
$h -> join_files('outfile.txt', 'FRET_MELT_DDGS.txt', 'FRET.txt');
```

Hoh.pm can also make Chimera attribute files to display data onto a protein structure. See <http://www.cgl.ucsf.edu/chimera/> for more details. Make a simple text file where the first column contains a unique list of residue numbers (i.e. 1, 2, 100, 101) or a list of residue types (i.e. GLY, LYS, PHE) followed by columns of data to be used to generate the attribute. Simply use the `save_as_chimera_attribute` method. Note the first argument must be a single word with not underscores or any punctuation, where the first letter is always lowercase (a weird Chimera issue); for example, 'phiValue' is good but not 'PhiValue' or 'Phi-value'! If you don't, it will create an appropriate name in the right format for you. The second argument contains the column name key to be exported; for these attributes only one column may be specified as the attribute:

```
use Hoh;
my $hoh_obj = Hoh -> new() -> open_hoh($hoh_file, $delimiter) ->
save_as_chimera_attribute($chimera_attribute_name, $col_with_attribute);
```

For example:

```
my $hoh_obj = Hoh->new()->open_hoh('PHICHIMERA.txt','') ->
save_as_chimera_attribute('phiValue2','PHI');
```

The Chimera attribute will be saved as a `yourChimeraAttributeName.txt` (whatever your name was) with the .txt extension so you may look at it easily in notepad or some simple text editor.

Here is an example of an output Chimera attribute file, called `phiValue.txt`:

```
# From Krantz Lab Hoh.pm
# Chimera attribute output file for residues
# Text file is PHICHIMERA.txt
attribute: phiValue
match mode: 1-to-1
recipient: residues
:39    0.5707
:40    -0.56007
:62    0.23958
:70    0.93064
:112   0.37229
:119   1.59486
:126   0.31066
:129   2.52437
:145   0.32538
:147   0.49904
:148   0.26554
:154   3.28114
:155   0.09889
:174   0.01373
:213   0.5012
:215   0.13245
:217   0.00406
:220   0.14255
:221   0.34891
:245   0.01833
:247   0.37886
```

...Happy hohing...